Project Workflow: Conditional GAN Augmentation for Imbalanced Skin Lesion Classification

Step-by-step execution guide for the DCGAN vs WGAN-GP comparison on HAM10000. Prepared for @Bhanu, Oct 22, 2026.

0. How to use this workflow

This document turns your project plan into an ordered checklist. Each phase has a goal, the exact steps, what you must produce, and a **gate**: a condition that must be true before you move on. The golden rules from your plan apply throughout:

1. **Only the training data changes.** Same ResNet18, same starting weights, same split, same optimiser, same epochs, same seeds in every experiment.
2. **The test split is sacred.** It never touches GAN training, classifier training, early stopping or model selection.
3. **Core first, extensions later.** E0, E2, E3, E4 must finish before anything else starts. If you fall behind, drop extensions, never core runs.
4. **Test the whole pipeline end to end on a tiny run before any long run.**
5. **A negative result is valid** if it is measured properly.

The four core experiments

ID	Training data	Answers
E0	Real only	Baseline: how bad is the imbalance?
E2	Real + classical augmentation on rare classes	Do flips/rotations/jitter help alone?
E3	Real + conditional DCGAN images	RQ1: do GAN images help?
E4	Real + conditional WGAN-GP images	RQ2: is Wasserstein loss better?

Extensions (only after core): E1 class weighting, E5 best GAN + classical, E6 top-up sweep (0/25/50/100%), E7 synthetic-only training.

---

1. Phase 1: Environment and repository setup (Day 1)

Goal: a reproducible project skeleton so every later run is repeatable.

Steps

1. Create a GitHub repo (or a Drive folder) with this structure:

```
project/
  data/            raw HAM10000 images + metadata csv (not committed)
  cache/           ham64.npz, splits.csv, synthetic_*.npz
  src/
    data.py        loading, splitting, preprocessing, datasets
    models.py      generator, discriminator, critic, classifier
    train_gan.py   DCGAN and WGAN-GP training loops
    train_clf.py   classifier training + evaluation
    metrics.py     macro-F1, recall, AUC, FID, nearest-neighbour
    utils.py       seeding, logging, checkpoint helpers
  configs/         one yaml per experiment (e0.yaml, e2.yaml, ...)
  checkpoints/     GAN and classifier checkpoints (sync to Drive)
  results/         csv of metrics, confusion matrices, image grids
  notebooks/       Colab/Kaggle runners
```

2. Install dependencies: `torch torchvision scikit-learn pandas numpy matplotlib seaborn torch-fidelity` (or `pytorch-fid`), plus optional `wandb` or `mlflow`.
3. Write `utils.py` with a `set_seed(seed)` function that seeds Python, NumPy and PyTorch (CPU and CUDA) and sets `torch.backends.cudnn.deterministic = True`.
4. Create one **config file per experiment** (yaml or a Python dict) holding every setting: seed, learning rate, epochs, batch size, input size, top-up target, GAN checkpoint path. Log the config next to each result. This is how you prove only the data changed.
5. On Colab, mount Google Drive and point `checkpoints/` and `results/` at Drive so session disconnects don't cost you work.

Gate 1: the repo runs `import src.*` without errors and `set_seed(0)` gives identical random numbers on two runs.

---

2. Phase 2: Data preparation and splits (Days 1 to 3)

Goal: one cached 64x64 array and a leakage-free, stratified split.

Steps

1. **Download HAM10000** from Kaggle or Harvard Dataverse. You need the images and `HAM10000_metadata.csv` (columns include `lesion_id`, `image_id`, `dx`).
2. **Map labels:** encode `dx` to integers in a fixed order: `akiec, bcc, bkl, df, mel, nv, vasc`. Save this mapping, because the conditional GAN and classifier must use the same label IDs.

3. **Split by `lesion_id`, not by image.** Use `StratifiedGroupKFold` (or a manual group split) with `groups=lesion_id` and `y=dx`:
   - Target: 70% train, 15% validation, 15% test.
   - Do it once, with a fixed seed, and save the result to `cache/splits.csv` (`image_id, lesion_id, dx, split`).
   - Because exact stratification is hard with groups, check the resulting class proportions per split and accept small deviations. Rare classes must appear in all three splits.
4. **Verify no leakage:** assert that the set of `lesion_id` values in train, val and test are pairwise disjoint.
5. **Preprocess every image once:**
   - Centre-crop 600x450 to 450x450, resize to 64x64 (use area/antialiased interpolation).
   - Store as `uint8` array of shape `(N, 64, 64, 3)` in `cache/ham64.npz` along with labels and split flags.
   - Scale to [-1, 1] **at load time** for the GAN (`x/127.5 - 1`).
   - For the classifier use the same 64x64 images, upsampled on the fly to the network input (e.g. 224 or 128) with ImageNet normalisation. Pick the input size once and never change it.
6. **Record the training counts per class.** Expect roughly: nv ~4,690, mel ~780, bkl ~770, bcc ~360, akiec ~230, vasc ~100, df ~80. These numbers decide your top-up targets later.
7. Save a grid of 8 real images per class from the training split as a sanity check (colours, orientation, labels correct).

Gate 2: splits are saved, leakage assertion passes, class table is recorded, and the sample grid looks right.

---

3. Phase 3: Classifier pipeline and E0 baseline (Days 3 to 6)

Goal: a fixed, trusted measuring instrument, and the baseline number every later result is compared with.

Steps

1. **Dataset class:** returns `(image_tensor, label)` from the cached array for a given split. Add a flag for optional classical augmentation (used only by E2) and a way to append extra `(image, label)` pairs (synthetic images, used by E3/E4).

2. **Model:** `torchvision.models.resnet18(weights='IMAGENET1K_V1')`, replace `fc` with `Linear(512, 7)`. Fine-tune all layers.
3. **Fixed training settings for every experiment:** cross-entropy loss, Adam lr 1e-4, batch size fixed (e.g. 64), 15 to 20 epochs, early stopping on **validation macro-F1** (patience ~5), keep the best-validation checkpoint.
4. **Do not change the sampler.** Use plain shuffling for all experiments, so the only difference is the data.
5. **Evaluation function** (`metrics.py`) that takes a trained model and the test split and returns:
   - Macro-F1 (primary), accuracy (reported only), balanced accuracy
   - Per-class recall (all 7 classes; mel, bcc and akiec highlighted)
   - Macro one-vs-rest AUC-ROC
   - Confusion matrix (save as csv and png)
6. **Run E0 with 3 seeds** (e.g. 0, 1, 2) and save a row per seed to `results/metrics.csv` with columns `experiment, seed, macro_f1, accuracy, bal_acc, auc, recall_akiec, ..., recall_vasc`.
7. **Sanity checks:** accuracy should be high (about 75 to 80%) but macro-F1 and rare-class recall noticeably lower. If macro-F1 is already near accuracy, something is wrong (check labels and split).
8. **Pipeline smoke test:** run the whole chain on a tiny subset (a few hundred images, 1 epoch, 2 GAN epochs) to confirm each stage hands its output to the next: split, GAN, generated array, classifier, metrics.

Gate 3: E0 gives sensible numbers with 3 seeds and a recorded standard deviation. The standard deviation is your noise floor: later gains smaller than this are not results.

---

4. Phase 4: Conditional DCGAN (Days 5 to 9)

Goal: a stable class-conditional GAN producing recognisable 64x64 lesion images.

Architecture

- **Generator:** noise `z` (100-d) concatenated with a learned class embedding (e.g. 50-d from `nn.Embedding(7, 50)`), linear projection to a 4x4x512 map, then four `ConvTranspose2d` blocks (4 to 8 to 16 to 32 to 64), each with BatchNorm and ReLU, final layer Tanh to 3 channels.
- **Discriminator:** embed the label to a 64x64 map (embedding, linear, reshape to 1 x 64 x 64) and concatenate with the image as a 4th channel. Four strided `Conv2d` blocks with LeakyReLU(0.2) (BatchNorm on all but the first), then a final conv/linear layer to a

single logit.
- Initialise weights from N(0, 0.02).

Training recipe

1. Train on the **training split only**, all 7 classes, images scaled to [-1, 1].
2. Optimiser: Adam, lr 2e-4, beta1 0.5, beta2 0.999, batch size 64.
3. **Class-balanced sampler:** `WeightedRandomSampler` with weight `1 / class_count` per sample, so rare classes appear often in each batch (this is for GAN training only, never the classifier).
4. **One-sided label smoothing:** real label 0.9, fake label 0.0.
5. Loss: binary cross-entropy (use `BCEWithLogitsLoss` for stability). Per step: update D on a real batch and a fake batch (fake labels sampled from the same balanced distribution), then update G to make D label fakes as real.
6. Train for roughly 100 to 300 epochs. **Time one epoch first** and scale to your 1 to 3 hour budget.
7. Every 5 to 10 epochs: save a checkpoint and an image grid (8 images per class from a **fixed noise vector**, so you can watch progress). Log D loss, G loss and D(real)/D(fake) averages.

What to watch

Symptom	Meaning	Fix
D loss near 0, G loss climbing	D overpowers G	Lower D lr, add label smoothing/noise, add DiffAugment
All images in a class look the same	Mode collapse	Balanced sampler, DiffAugment, or move on to WGAN-GP
Blobs with no structure after many epochs	Underfit/bad init	Check normalisation range and Tanh, train longer

8. If D overfits on the tiny classes, add **DiffAugment** (translation, colour, cutout) applied to both real and fake images before D, per Zhao et al.
9. Select the checkpoint using visual grids plus FID (Phase 6), not the final epoch by default.

Gate 4 (end of Day 9): DCGAN produces recognisable lesion-like images for most classes. If it is badly unstable, document it and rely on WGAN-GP, but still use the best DCGAN checkpoint for E3, because the comparison is the contribution.

---

5. Phase 5: Conditional WGAN-GP (Days 7 to 12)

Goal: a stable alternative using Wasserstein loss, trained on identical data.

Start this as soon as the DCGAN pipeline runs. The two trainings overlap on purpose.

Architecture changes

- **Generator:** identical to the DCGAN generator (keeps the comparison about the loss, not the network).
- **Critic:** same convolutional layout as the discriminator but **no sigmoid and no BatchNorm**. Use `LayerNorm` (GroupNorm with 1 group) or `InstanceNorm2d`. Outputs an unbounded scalar. The class label is injected the same way as in the discriminator.

Training recipe

1. Adam, lr 1e-4, betas (0.0, 0.9), batch size 64, class-balanced sampler (same as DCGAN).
2. **5 critic updates per generator update.**
3. Critic loss: `D(fake|y) - D(real|y) + 10 * GP`.
4. **Gradient penalty:**
   - Sample `eps ~ U(0,1)` per image, interpolate `x_hat = eps * real + (1 - eps) * fake`.
   - Set `x_hat.requires_grad_(True)`, compute `D(x_hat | y)`.
   - Get gradients with `torch.autograd.grad(outputs, x_hat, grad_outputs=ones, create_graph=True)`.
   - Penalty = mean of `(grad_norm_per_sample - 1)^2` where the norm is over all pixel dimensions.
5. Generator loss: `-D(G(z, y) | y).mean()`.
6. Because there are 5 critic steps per generator step, the same number of generator iterations takes longer. Match the training budget to DCGAN by total **real images seen** or wall-clock time, and state which you used in the report.
7. Log critic loss, Wasserstein estimate (`D(real) - D(fake)`), and gradient-norm average (should hover near 1). Save checkpoints and fixed-noise grids as before.

What to watch

- Wasserstein estimate should shrink and flatten as training improves; wild oscillation suggests too high lr or a bug in the penalty.
- Gradient norms far from 1 mean the penalty is not being applied correctly (check you do not use BatchNorm in the critic).

Gate 5 (end of Day 12): WGAN-GP produces recognisable images and a saved best checkpoint exists.

---

6. Phase 6: Generating synthetic images, FID and memorisation checks (Days 11 to 13)

Goal: the synthetic sets for E3 and E4, plus the GAN-quality evidence. **Synthetic images must exist by end of Day 13.**

Step A: FID

1. For each GAN, generate **at least 1,000 images per class** (class-conditioned) at 64x64.
2. Compute FID with `torch-fidelity` or `pytorch-fid` against **real training images of the same class**, plus an overall FID against all real training images. Use identical settings (same image preprocessing and Inception weights) for both GANs.
3. Save a table: rows = classes + overall, columns = DCGAN, WGAN-GP.
4. Interpret carefully: with only 80 to 360 real training images per rare class, per-class FID is noisy. Use it to **compare the two GANs**, never as an absolute quality score, and say so in the report.

Step B: Visual check

For each rare class, make a grid with real images in one row and DCGAN and WGAN-GP samples in the next rows. Include these in the report.

Step C: Memorisation check

1. Extract features for real training images and synthetic images using the same pretrained ResNet18 (penultimate layer) or Inception.
2. For each synthetic image, find the nearest real training image of the same class (cosine or L2 distance).
3. Plot the distribution of nearest-neighbour distances and show a few side-by-side pairs of synthetic vs nearest real. Distances close to zero indicate copying.
4. Compare with the real-to-real baseline: nearest neighbour distance between held-out real images and the training set.

Step D: Build the synthetic sets

1. Choose the **minority classes to top up**: `akiec, vasc, df` (and optionally `bcc`). Do **not** generate for `mel` or `nv`, but keep `mel` and `bcc` in all result tables and confusion matrices.

2. Choose the **top-up target** for the core run: e.g. bring each rare class up to 50% of the largest rare class (largest rare class here is `bcc` at about 360 if included, or `akiec` at about 230 otherwise). Fix this number now and use it for both GANs. Write down the rule.
3. Optional quality filter: score synthetic images with the E0 classifier and drop those classified as the wrong class with high confidence (e.g. confidence > 0.9). Apply the **same filter rule** to both GANs and report how many were removed.
4. Generate surplus images (e.g. 2x the target) so filtering still leaves enough, then trim to exactly the target count per class.
5. Save as `cache/synthetic_dcgan.npz` and `cache/synthetic_wgangp.npz` (uint8 images and labels). Save a grid of the final sets.

Gate 6 (end of Day 13): both synthetic files exist with identical class counts, FID table and NN plots are saved.

---

7. Phase 7: Core classifier runs E2, E3, E4 (Days 12 to 17)

Goal: the headline table.

For each experiment, run **3 seeds** with the exact E0 settings.

Run	What changes in the training set	Notes
E2	Rare classes (same ones as the GAN top-up) get classical augmentation: flips, rotations, colour jitter, to reach the same target count	Augment **on the fly** or pre-generate to the same count, so the number of extra samples equals the GAN runs
E3	E0 data + DCGAN synthetic set	Synthetic images added to train only
E4	E0 data + WGAN-GP synthetic set	Identical count, filter and classifier settings

Steps

1. Load the real training set; append the synthetic arrays (labels attached). Validation and test stay **real only**.
2. Convert synthetic images exactly as real ones (same 64x64, same upsampling and normalisation) so the classifier cannot tell them apart by resolution.
3. Train from the **same ImageNet starting weights** with the same hyperparameters, early stopping on real validation macro-F1.
4. Evaluate on the real test split and append metrics to `results/metrics.csv`.

5. Produce the summary table: mean and standard deviation across seeds for macro-F1, balanced accuracy, AUC and per-class recall for E0, E2, E3, E4.
6. **Decision rule:** call a difference real only if the gap in mean macro-F1 is larger than the seed standard deviation. If it is not, report "no measurable gain".
7. Optional but cheap: run a paired comparison (same seeds) or a bootstrap confidence interval over test predictions to back the claim.

Gate 7 (end of Day 17): the headline table for E0, E2, E3, E4 is complete with 3 seeds each. Everything after this point is bonus.

---

8. Phase 8: Extensions (Days 16 to 19, only if core is complete)

Start extensions in this priority order:

1. **E6, top-up sweep** (0%, 25%, 50%, 100%) using the better GAN. Plot macro-F1 and rare-class recall versus synthetic amount, with error bars. Answers RQ3 and powers the demo slider.
2. **E5, GAN + classical augmentation** (best GAN images plus E2 transforms). Answers RQ4.
3. **E1, class-weighted loss.** Weights inverse to training class frequency. Shows whether a loss tweak alone is enough.
4. **E7, synthetic-only rare-class images** (tested on real) to check that the images carry real class information.
5. **Grad-CAM** on rare-class predictions with and without GAN data (links to Unit 6).

Stop at Day 19 regardless of what has finished.

---

9. Phase 9: Analysis (Days 17 to 21)

Goal: turn numbers into answers to RQ1 to RQ4.

1. **Headline table:** E0, E2, E3, E4 (plus extensions) with mean ± std for macro-F1, per-class recall, balanced accuracy, AUC, and FID for the two GANs.
2. **Confusion matrices** before and after augmentation (E0 vs best run), normalised by true class. Focus on **mel vs nv** confusion and on the rare classes.
3. **Per-class recall bar chart** for E0, E2, E3, E4, with all 7 classes shown (mel and bcc must appear even though they were not generated).
4. **Check for side effects:** does adding rare-class synthetic images reduce nv or mel recall? Report that honestly, since a gain in rare classes that costs melanoma recall is clinically important.

5. **GAN comparison:** FID table, training-loss curves, image grids, memorisation plot, a short note on stability (did DCGAN collapse? how many restarts?).
6. **Answer each research question explicitly:**
   - RQ1: GAN images vs E0 on macro-F1.
   - RQ2: WGAN-GP vs DCGAN on FID and on classifier gain.
   - RQ3/RQ4: only if the extensions ran.
7. **Limitations to state:** 64x64 resolution loses fine dermoscopic detail; FID with Inception features is not a measure of medical realism; per-class FID is noisy; one dataset; this is a course prototype, not a diagnostic tool.

---

10. Phase 10: Report, slides and demo (Days 17 to 21)

Report outline

1. Introduction and motivation (class imbalance, cost of missed cancers)
2. Dataset and splits (lesion-level split, preprocessing, class table)
3. Methods (conditional DCGAN, conditional WGAN-GP with the loss and GP equation, ResNet18 setup, how synthetic images enter training)
4. Experiments (E0 to E4 plus extensions; what was held fixed)
5. Results (headline table, per-class recall, confusion matrices, FID, NN check, extension plots)
6. Discussion (answers to RQ1 to RQ4, why results came out this way, negative results)
7. Limitations and conclusion
8. References (**verify every reference** before citing; your plan notes they were listed from memory)

Demo (under two minutes)

Build a Streamlit or Gradio page with:

- A class selector that generates new lesion images from the saved generator.
- A side-by-side view: the same test image scored by the E0 model and the GAN-augmented model.
- If E6 ran, a slider showing rare-class recall versus number of synthetic images.

Pre-compute outputs where possible so the demo does not depend on a live GPU.

Viva preparation

Rehearse the questions in your plan, plus these: Why did you not generate melanoma? How do you know the GAN is not memorising? What would you change with more compute (higher resolution, StyleGAN2-ADA, diffusion)?

---

11. Master timeline with gates

Days	Task	Output	Gate
1	Setup	Repo, configs, seeding	Gate 1
1-3	Data and splits	ham64.npz, splits.csv	Gate 2
3-6	Pipeline test and E0	E0 x 3 seeds	Gate 3
5-9	Conditional DCGAN	Checkpoint, grids	Gate 4
7-12	Conditional WGAN-GP	Checkpoint, grids	Gate 5
11-13	Generate, FID, NN check	Synthetic sets	Gate 6 (day 13)
12-17	Core runs E2, E3, E4	Headline table	Gate 7
16-19	Extensions (if time)	E6, E5, E1	none
17-21	Analysis, report, slides, demo	Final submission	done

Weekly checkpoints

- **End of Week 1 (day 7):** splits done, full pipeline tested, E0 running, DCGAN making lesion-like images.
- **End of Week 2 (day 14):** WGAN-GP trained, FID for both, synthetic sets ready, first E3/E4 results.
- **End of Week 3 (day 21):** core table with 3 seeds, analysis, report, slides, demo.

If you fall behind

1. Drop E7, then E5, then E1, then E6.
2. Shorten GAN training (fewer epochs) before cutting seeds.
3. Reduce the classifier to 15 epochs.
4. Never cut E0, E2, E3 or E4, and never skip the lesion-level split.

---

12. Troubleshooting guide

Problem	Likely cause	What to do
Test scores suspiciously high	Lesion leakage across splits	Re-check group split and assert disjoint lesion IDs
E0 macro-F1 almost equals accuracy	Label mismatch or wrong metric averaging	Verify label mapping; use `average='macro'`
GAN images all one colour/blob	Wrong range or activation	Confirm data in [-1, 1], Tanh output, N(0, 0.02) init
DCGAN collapse on rare classes	D too strong	Label smoothing, DiffAugment, lower D lr, or rely on WGAN-GP
WGAN-GP loss explodes	BatchNorm in critic, or lr too high	Use LayerNorm/InstanceNorm, lr 1e-4, verify GP
Classifier gains from GAN data but test set not clean	Synthetic leaking into val/test	Add synthetic to train only
Classifier "detects" synthetic images	Resolution or processing mismatch	Process real and synthetic through the identical pipeline
Colab disconnects	Free tier limits	Checkpoint every few epochs to Drive, resume from checkpoint
Results vary wildly across seeds	High variance on tiny classes	Report std, add seeds if time allows, use bootstrap CIs
Every GAN number is worse than baseline	Poor image quality or label noise	Apply the quality filter, reduce top-up amount, report honestly

---

13. Final submission checklist

- [ ] Code runs in order: data prep, both GANs, generation, classifier, evaluation
- [ ] `splits.csv` and label mapping saved; no lesion appears in two splits
- [ ] Configs and seeds logged for every run
- [ ] E0, E2, E3, E4 each have 3 seeds in `metrics.csv`
- [ ] Results table: macro-F1, per-class recall (all 7 classes), balanced accuracy, AUC, mean ± std
- [ ] FID (overall and per class) for DCGAN and WGAN-GP with the caveats stated

- [ ] Loss curves and real vs generated grids for each rare class
- [ ] Nearest-neighbour memorisation check
- [ ] Confusion matrices before and after augmentation
- [ ] Extensions clearly labelled as done or not done
- [ ] References verified
- [ ] Report, slides and demo ready; limitations and "not a diagnostic tool" note included